



RESUMÃO PARA PROVA

Práticas de Programação (Python)

Capítulos 1 a 8 — com analogias do dia a dia + quiz de 50 questões e gabarito



Sumário

- Módulo 1 — Programação Estruturada
- Módulo 2 — Listas
- Módulo 3 — Fatiamento (Slicing)
- Módulo 4 — Tuplas e Dicionários
- Módulo 5 — Função Lambda
- Módulo 6 — Tratamento de Exceções
- Módulo 7 — Arquivos
- Módulo 8 — Gerenciamento de Módulos
- Quiz — 50 perguntas
- Gabarito

Módulo 1 — Programação Estruturada

Pontos-chave

- Programação estruturada = organizar o código em funções e procedimentos, em vez de um bloco único gigante.
- Função: bloco reutilizável que EXECUTA algo e RETORNA um valor (usa a palavra return).
- Procedimento: igual à função, mas não retorna valor explícito (sem return). Em Python, na prática, todo procedimento retorna None por baixo dos panos.
- Parâmetros default (ex: media_aprovacao=7.0): valor usado quando o argumento não é informado na chamada.
- Escopo global: variável declarada fora de funções, acessível em qualquer lugar do código.
- Escopo local: variável declarada dentro de uma função, só existe e só pode ser usada dentro dela.
- Usar variáveis globais em excesso é arriscado: dificulta debug, cria dependências ocultas e risco de sobrescrita acidental.
- Funções em Python são 'cidadãs de primeira classe' (first-class citizens): podem ser guardadas em variáveis, passadas como argumento e retornadas por outras funções — isso é a base do lambda, map, filter etc.

Código essencial

```
def calcular_media(n1, n2, n3):  
    return (n1 + n2 + n3) / 3  
  
def verificar_aprovacao(media, corte=7.0):  
    return "Aprovado" if media >= corte else "Reprovado"
```

Pensa numa cozinha de restaurante

Em vez de um cozinheiro só fazendo TUDO (entrada, prato principal, sobremesa) numa bagunça só, você divide: um cuida do prato principal (função calcular_media), outro do molho (verificar_aprovacao). Cada um tem sua bancada só dele (variável LOCAL) — mas o cardápio pregado na parede (variável GLOBAL) qualquer um pode ler. Isso é modularização: cada função com sua responsabilidade, sem bagunçar a cozinha toda.

Módulo 2 — Listas

Pontos-chave

- Estruturas unidimensionais armazenam dados em uma única dimensão, indexados a partir de 0 (o último índice é $n-1$).
- Homogênea: todos elementos do mesmo tipo. Heterogênea: tipos diferentes na mesma lista.
- list (Python) usa alocação DINÂMICA de memória (flexível, cresce/diminui). array.array usa memória CONTÍGUA (mais rápido para cálculos numéricos, mas só aceita um tipo de dado).
- Inserir: append() no final | insert(pos, valor) numa posição | extend([...]) adiciona vários itens de uma vez.
- Remover: remove(valor) tira a 1ª ocorrência | del lista[i] remove pelo índice | clear() esvazia tudo.
- Alterar: basta atribuir um novo valor pelo índice, ex: lista[2] = 35.
- Buscar: operador in (lista mais rápida/pythonica) ou loop manual percorrendo índice a índice.
- Listas são passadas por REFERÊNCIA para funções: se a função modifica a lista dentro dela, a lista original também muda! Para evitar isso, use .copy().
- Estruturas bidimensionais (matrizes) em Python = listas dentro de listas (listas aninhadas). Acesso: matriz[linha][coluna]. Percorrer exige DOIS loops (um dentro do outro).

Código essencial

```
def adicionar_sem_alterar(lista):  
    nova = lista.copy()    # desvincula da original  
    nova.append(100)  
    return nova
```

Lista é tipo um prédio de apartamentos

Cada apartamento tem um número (índice), começando do térreo = 0. Se você quer o morador do 3º apê, é lista[3]. Uma matriz (lista de listas) é tipo um condomínio com vários prédios: primeiro você escolhe o prédio (linha), depois o apartamento (coluna) — matriz[prédio][apê]. E cuidado: se você empresta a 'planta do prédio' (a lista) pra alguém mexer sem fazer cópia, e essa pessoa reforma um apê, a reforma valeu pro prédio de verdade também — porque é a MESMA referência, não uma cópia!

Módulo 3 — Fatiamento (Slicing)

Pontos-chave

- Sintaxe: `lista[início:fim:passo]` — início é INCLUSIVO, fim é EXCLUSIVO, passo é o intervalo entre índices (padrão: 0, tamanho da lista, 1).
- `numeros[2:7]` pega do índice 2 até o 6 (o índice 7 fica de fora).
- Passo: `numeros[::2]` pega de 2 em 2 (pares de índice). Passo negativo `numeros[::-1]` inverte a lista/string inteira.
- O fatiamento SEMPRE cria uma NOVA lista/string, independente da original (cuidado com uso de memória em listas muito grandes).
- Strings também aceitam fatiamento (são tratadas como uma 'lista de caracteres'), mas são IMUTÁVEIS — não dá pra alterar um caractere direto pelo índice.
- Strings: concatenação com `+` | repetição com `*` | `len()` conta caracteres | `in` / `not in` verificam se uma substring está contida.
- `s[::-1]` é o truque clássico para inverter string / checar palíndromo.

Código essencial

```
numeros = [0,1,2,3,4,5,6,7,8,9]
sublista = numeros[2:7]      # [2, 3, 4, 5, 6]
invertida = numeros[::-1]    # [9,8,7,6,5,4,3,2,1,0]

s = "Python"
print(s[::-1])               # nohtyP
```

É tipo cortar um pedaço de um rolo de filme fotográfico

lista[início:fim:passo] é você dizendo 'corta a partir daqui (início, incluído), até ANTES daqui (fim, excluído), pulando de tantos em tantos quadros (passo)'. E o resultado do corte é sempre um pedaço NOVO e separado — o filme original continua intacto, do jeitinho que estava.

Módulo 4 — Tuplas e Dicionários

Pontos-chave

- Tupla: estrutura ORDENADA e IMUTÁVEL — depois de criada, não muda. Sintaxe: `minha_tupla = (1, 2, 3)`.
- Ideal para dados em que a ORDEM importa e não deve mudar: coordenadas (latitude, longitude), cores RGB/CMYK, registros fixos.
- Dicionário: estrutura de CHAVE-VALOR (como um mapa/tabela de consulta), MUTÁVEL. Sintaxe: `{}` ou `dict()`.
- Acesso ao valor: `dicionario['chave']`. Dicionários podem ser aninhados (dicionário dentro de dicionário).
- Atualizar: `dicionario['chave'] = novo_valor` (atualiza se existe, cria se não existe).
- Remover: `del dicionario['chave']` remove | `.pop('chave')` remove E retorna o valor removido.
- Métodos nativos: `.keys()` (só chaves), `.values()` (só valores), `.items()` (pares chave-valor, útil pra percorrer com `for` `chave, valor in dic.items()`).
- Tuplas passadas para função: os dados NÃO podem ser alterados dentro da função (proteção total).
- Dicionários passados para função são por REFERÊNCIA (igual lista) — a função pode alterar o dicionário original! Use `.copy()` para evitar isso.
- Dicionários combinam muito bem com JSON/XML (formatos de troca de dados entre sistemas).

Código essencial

```
peessoa = {"nome": "João", "idade": 35}
for chave, valor in peessoa.items():
    print(f"{chave}: {valor}")

coordenada = (-23.66, -46.70) # tupla: ordem fixa, não muda
```



Tupla é a data do seu aniversário, dicionário é sua agenda de contatos

Sua data de nascimento (dia, mês, ano) é uma TUPLA: a ordem é sagrada e nunca muda, senão vira outra data. Já a agenda de contatos do celular é um DICIONÁRIO: você procura pelo NOME (chave) e acha o telefone (valor), e pode editar, apagar ou adicionar contato à vontade — porque é mutável.

Módulo 5 — Função Lambda

Pontos-chave

- Lambda = função anônima de UMA LINHA SÓ, sem nome formal (vem do 'cálculo lambda' de Alonzo Church, 1930s).
- Sintaxe: lambda parametros: expressão — a expressão já É o retorno (não usa a palavra return).
- Serve para operações curtas e descartáveis, usadas só uma vez dentro de outra expressão. Para lógica mais longa/complexa, use def normal (mais legível).
- Lambda NÃO é mais rápida que def — o ganho é em concisão de código, não em performance.
- sort(key=lambda x: ...) / sorted(lista, key=lambda x: ...): ordena definindo um critério customizado (ex: ordenar tuplas pelo 2º elemento).
- map(função, iterável): aplica uma função a CADA item de uma lista/tupla, retorna um iterador (precisa envolver em list() para virar lista de novo).
- list(): converte qualquer iterável (range, map, filter, zip...) em uma lista de verdade.
- Combo clássico de prova: list(map(lambda x: ..., lista)) — substitui um for + append inteiro em uma linha.

Código essencial

```
celsius = [0, 10, 20, 30, 40, 100]
fahrenheit = list(map(lambda c: (c*9/5)+32, celsius))

pessoas = [("Alice",25), ("Bob",30), ("Carol",20)]
pessoas.sort(key=lambda x: x[1]) # ordena pela idade
```

Lambda é tipo um post-it, def é tipo um documento oficial

Uma função normal (def) é um documento formal, com nome, arquivado, feito pra ser usado várias vezes. Uma lambda é aquele post-it rápido que você cola, usa uma vez pra resolver algo pontual ('multiplica por 2 aqui') e depois já era — não precisa nem guardar. Ótimo pra recado rápido, péssimo pra virar 'documento oficial' de algo complexo.

Módulo 6 — Tratamento de Exceções

Pontos-chave

- Erro de SINTAXE ≠ EXCEÇÃO. Erro de sintaxe é código mal escrito (nem roda). Exceção é um problema que acontece em TEMPO DE EXECUÇÃO, mesmo com o código certo.
- Estrutura básica: try (bloco que pode falhar) / except TipoDeErro (o que fazer se falhar).
- Dá pra ter vários except's para tratar tipos de erro diferentes separadamente (ex: ValueError vs ZeroDivisionError).
- else: roda só se NÃO houver nenhuma exceção. finally: roda SEMPRE, tenha dado erro ou não (ótimo para 'limpar a casa', fechar conexões etc.).
- Principais exceções: ZeroDivisionError (divisão por zero), ValueError (valor inadequado), TypeError (tipo errado), IndexError (índice fora do intervalo), KeyError (chave não existe no dicionário), FileNotFoundError (arquivo não encontrado).
- raise: dispara uma exceção MANUALMENTE, mesmo sem erro nativo do Python (ex: validar regra de negócio como idade mínima).
- Boas práticas: evitar 'except Exception' genérico demais (mascara erros), preferir capturar exceções específicas, e usar o módulo logging em vez de print() para registrar erros de forma estruturada.

Código essencial

```
try:
    idade = int(input("Digite a idade: "))
    if idade < 18:
        raise ValueError("Menor de idade")
except ValueError as e:
    print(f"Erro: {e}")
finally:
    print("Processo encerrado")
```



É EXATAMENTE tipo o celular tocando numa reunião

Uma exceção é como o celular tocando no meio de uma reunião importante (o programa rodando). Sem tratamento (sem try/except), a reunião simplesmente PARA e todo mundo vai embora (o programa quebra/crasha). Com o try/except, você trata a interrupção com jeito: atende rapidinho, resolve, e a reunião CONTINUA de onde parou. O finally é tipo aquele aviso que você sempre dá no fim, chova ou faça sol: 'reunião encerrada' — acontece do mesmo jeito, com ou sem interrupção no meio.

Módulo 7 — Arquivos

Pontos-chave

- Codificação de caracteres: UTF-8 (padrão web/Python, compatível com ASCII) é a mais usada; UTF-16 comum no Windows/sistemas asiáticos; ASCII só 128 caracteres básicos.
- `open('arquivo', 'r')` abre o arquivo, MAS precisa fechar manualmente com `.close()` no final.
- `with open('arquivo', 'r')` as f: é o jeito recomendado — fecha o arquivo AUTOMATICAMENTE ao sair do bloco, mesmo se der erro no meio.
- Modos de abertura: 'r' leitura (erro se não existir) | 'w' escrita (cria ou APAGA e sobrescreve) | 'a' adiciona no final (append, sem apagar) | 'x' cria (erro se já existir) | 'b' modo binário | '+' leitura E escrita juntas.
- Leitura: `.read()` lê tudo | `.readline()` lê uma linha por vez | `.readlines()` lê tudo já organizado em lista de linhas.
- Escrita: `.write()` escreve texto | `.writelines()` escreve uma lista de linhas.
- Arquivos binários (imagens, áudio, vídeo, executáveis): abrir com 'rb'/'wb', sem decodificação automática, manipulados em bytes.
- Exceções comuns de arquivo: `FileNotFoundError`, `PermissionError`, `IsADirectoryError`, `UnicodeDecodeError` (codificação errada na leitura).
- Operador walrus (`:=`) permite atribuir e testar um valor na mesma expressão — muito usado em `while` (linha `:= arquivo.readline()`): para ler até o fim do arquivo (EOF).

Código essencial

```
with open("exemplo.txt", "r", encoding="utf-8") as arquivo:
    conteudo = arquivo.read()

with open("saida.txt", "a") as arquivo:
    arquivo.write("nova linha\n")
```



Arquivo texto é a gaveta de papéis, arquivo binário é o cofre

Um arquivo *TEXTO* é tipo uma gaveta de papéis: qualquer pessoa abre e lê o conteúdo direto, sem ferramenta especial (por isso a preocupação com a 'língua'/codificação em que está escrito). Um arquivo *BINÁRIO* é tipo um cofre trancado: você só entende o conteúdo com o programa certo (leitor de imagem, player de vídeo), porque por dentro é só uma sequência de bytes crus. E o 'with open(...) as' é você contratando alguém que promete: abrir a gaveta, você mexer, e ELE fecha sozinho depois — você não corre o risco de esquecer a gaveta aberta.

Módulo 8 — Gerenciamento de Módulos

Pontos-chave

- Python foi criado por Guido van Rossum (1989). PyPI (Python Package Index) é o repositório oficial de pacotes/bibliotecas Python.
- pip (Pip Installs Packages) é o gerenciador de pacotes padrão do Python (a partir da versão 3.4).
- Comandos principais: `pip install pacote` | `pip install pacote==versão` | `pip uninstall pacote` | `pip list` (lista instalados) | `pip show pacote` (detalhes) | `pip list --outdated` (desatualizados) | `pip install --upgrade pacote`.
- `pip freeze > requirements.txt`: gera um arquivo com todas as dependências do projeto e suas versões exatas.
- `pip install -r requirements.txt`: instala tudo que está listado no requirements.txt (reproduz o ambiente em outra máquina).
- Ambiente virtual (venv): isola as dependências de CADA projeto, evitando conflito de versões entre projetos diferentes.
- Criar: `python -m venv nome_ambiente` | Ativar (Linux/Mac): `source nome_ambiente/bin/activate` | Ativar (Windows CMD): `nome_ambiente\Scripts\activate` | Desativar: `deactivate`.
- Estrutura mínima de um pacote próprio: pasta com `__init__.py` (marca que é um pacote), `setup.py` ou `pyproject.toml` (metadados/instalação), `README.md`, `LICENSE`.
- Publicar pacote: `python -m build` (gera o pacote) → `twine upload dist/*` (envia pro PyPI).

Código essencial

```
python -m venv meu_ambiente
source meu_ambiente/bin/activate      # Linux/Mac
pip install requests
pip freeze > requirements.txt
```



pip é a loja de aplicativos, venv é ter um celular separado por projeto

O pip é tipo a App Store/Play Store do Python: você instala, atualiza e desinstala 'apps' (pacotes) que outras pessoas fizeram. O requirements.txt é a lista de apps que você tem instalado, pra 'clonar' o mesmo celular em outro aparelho. E o ambiente virtual (venv) é como ter um celular SEPARADO só pra cada projeto — assim um app de um projeto não bagunça ou entra em conflito com o app (ou a versão do app) que outro projeto está usando.



QUIZ — 50 Perguntas

Responda sem consultar o resumo. O gabarito está na última seção do documento. Bora testar o que já ficou na cabeça!



Módulo 1 — Programação Estruturada

1. Qual a principal diferença conceitual entre uma função e um procedimento em Python?
 - a) Função aceita parâmetros, procedimento não
 - b) Função retorna um valor com 'return', procedimento não retorna valor explícito
 - c) Procedimento só pode ser chamado uma vez
 - d) Não existe diferença, são sinônimos
2. O que caracteriza uma variável de escopo LOCAL?
 - a) É acessível em qualquer parte do programa
 - b) Só existe e pode ser usada dentro do bloco/função onde foi declarada
 - c) Só pode guardar números
 - d) É sempre declarada no início do arquivo
3. Em 'def verificar_aprovacao(media, media_aprovacao=7.0):', o que representa o '=7.0'?
 - a) Um erro de sintaxe
 - b) Um parâmetro default (valor padrão usado se não for informado)
 - c) Uma variável global obrigatória
 - d) O tipo de dado do parâmetro
4. Por que dizemos que funções em Python são 'cidadãs de primeira classe'?
 - a) Porque só podem ser usadas por programadores experientes
 - b) Porque podem ser atribuídas a variáveis, passadas como argumento e retornadas por outras funções
 - c) Porque rodam mais rápido que procedimentos
 - d) Porque não podem receber parâmetros
5. Qual é um risco do uso excessivo de variáveis GLOBAIS?
 - a) O código fica mais rápido
 - b) Dificulta a depuração e cria dependências ocultas entre partes do código
 - c) É proibido pela linguagem Python
 - d) Reduz o consumo de memória
6. Qual é a principal vantagem de dividir um programa em funções/procedimentos (programação estruturada)?
 - a) O código fica maior
 - b) Organização, reutilização e facilidade de manutenção do código
 - c) Elimina a necessidade de variáveis
 - d) Torna o Python mais lento

Módulo 2 — Listas

7. Em uma lista Python, qual é o índice do primeiro elemento?
 - a) 1
 - b) -1
 - c) 0
 - d) Depende do tamanho da lista
8. Qual método adiciona um único elemento ao FINAL de uma lista?
 - a) insert()

- b) `append()`
- c) `extend()`
- d) `add()`

9. Qual método remove a PRIMEIRA ocorrência de um valor específico em uma lista?

- a) `del`
- b) `clear()`
- c) `remove()`
- d) `pop()`

10. Qual a diferença principal entre a estrutura 'list' e 'array.array' em Python?

- a) list só aceita números, array aceita qualquer tipo
- b) array.array armazena dados de forma contígua e homogênea, com melhor desempenho numérico; list é mais flexível mas menos otimizada para cálculos
- c) Não há diferença prática
- d) array.array não existe em Python

11. Como uma matriz (estrutura bidimensional) costuma ser representada em Python?

- a) Com o tipo nativo 'matrix'
- b) Através de listas aninhadas (listas dentro de listas)
- c) Só é possível com bibliotecas externas
- d) Com dicionários obrigatoriamente

12. Ao passar uma lista como argumento para uma função sem usar `.copy()`, o que acontece se a função modificar a lista internamente?

- a) A lista original NÃO é afetada, pois listas são sempre copiadas
- b) A lista original também é modificada, pois listas são passadas por referência
- c) Gera um erro de tipo (TypeError)
- d) A função cria uma nova lista automaticamente

13. Para percorrer todos os elementos de uma matriz (lista de listas), o que geralmente é necessário?

- a) Um único loop for
- b) Dois loops for aninhados (um para linha, outro para coluna)
- c) Não é possível percorrer uma matriz
- d) Usar apenas o método `.sort()`

Módulo 3 — Fatiamento (Slicing)

14. Na sintaxe `lista[início:fim:passo]`, o índice 'fim' é:

- a) Inclusivo (o elemento nesse índice entra no resultado)
- b) Exclusivo (o elemento nesse índice NÃO entra no resultado)
- c) Sempre obrigatório
- d) Igual ao índice 'início'

15. O que retorna `numeros[::-1]`, sendo `numeros = [1,2,3,4,5]`?

- a) `[1,2,3,4,5]`
- b) `[5,4,3,2,1]`
- c) Um erro
- d) `[1,3,5]`

16. Qual é o resultado de `'Python'[0:4]`?

- a) 'Pyth'
- b) 'ython'
- c) 'Python'
- d) 'th'

17. Ao fatiar uma lista (ex: lista[2:5]), o que é gerado?

- a) Uma referência para a lista original
- b) Uma NOVA lista, independente da original
- c) Um erro, pois listas não podem ser fatiadas
- d) A lista original é apagada

18. Por que strings em Python são consideradas imutáveis?

- a) Porque não podem ser fatiadas
- b) Porque, uma vez criadas, seu conteúdo não pode ser alterado diretamente pelo índice
- c) Porque não aceitam concatenação
- d) Porque só armazenam números

19. Qual operador é usado para verificar se uma substring está contida em uma string?

- a) contains()
- b) has
- c) in
- d) ==

Módulo 4 — Tuplas e Dicionários

20. Qual a principal característica de uma TUPLA em Python?

- a) É mutável e desordenada
- b) É imutável (não pode ser alterada após criada) e ordenada
- c) Só armazena números
- d) É idêntica a um dicionário

21. Como se acessa o valor associado à chave 'nome' em um dicionário chamado 'pessoa'?

- a) pessoa(nome)
- b) pessoa['nome']
- c) pessoa->nome
- d) pessoa.nome()

22. Qual método remove uma chave de um dicionário E retorna o valor removido?

- a) del
- b) remove()
- c) pop()
- d) clear()

23. Qual método retorna todos os pares chave-valor de um dicionário, útil para uso em um loop for?

- a) .keys()
- b) .values()
- c) .items()
- d) .pairs()

24. Quando um dicionário é passado como argumento para uma função, ele é passado:

- a) Por valor (sempre uma cópia)
- b) Por referência, podendo ser alterado dentro da função e refletir na origem
- c) De forma criptografada
- d) Apenas como somente-leitura

25. Por que tuplas são úteis para representar coordenadas geográficas (latitude, longitude)?

- a) Porque são mutáveis e fáceis de editar
- b) Porque a ORDEM dos dados é fixa e não pode ser alterada acidentalmente
- c) Porque ocupam mais memória que listas
- d) Porque aceitam apenas texto

26. Se uma função recebe uma tupla como parâmetro e tenta modificar um de seus valores, o que ocorre?

- a) A modificação funciona normalmente
- b) Ocorre um erro, pois tuplas são imutáveis
- c) A tupla vira automaticamente uma lista
- d) Nada acontece, o valor simplesmente é ignorado

Módulo 5 — Função Lambda

27. Qual é a sintaxe correta de uma função lambda que soma dois números?

- a) `lambda x, y: return x + y`
- b) `lambda x, y: x + y`
- c) `def lambda(x, y): x + y`
- d) `lambda(x, y) => x + y`

28. Funções lambda são mais indicadas para:

- a) Lógicas longas e complexas com várias linhas
- b) Operações curtas e descartáveis, usadas uma única vez
- c) Substituir totalmente as funções `def` no projeto
- d) Apenas trabalhar com arquivos

29. O que a função `map()` faz?

- a) Ordena uma lista
- b) Aplica uma função a cada item de um iterável, retornando um iterador
- c) Remove elementos duplicados
- d) Cria um dicionário a partir de uma lista

30. Por que geralmente envolvemos o resultado de `map()` com `list()`?

- a) Porque `map()` retorna um iterador, e `list()` converte esse iterador em uma lista de fato
- b) Porque é obrigatório por sintaxe
- c) Porque `map()` sempre gera erro sem `list()`
- d) Porque `list()` ordena o resultado

31. Em termos de desempenho, uma função lambda comparada a uma função `def` equivalente é:

- a) Sempre mais rápida
- b) Sempre mais lenta
- c) Sem vantagem significativa de performance — a diferença está na concisão do código
- d) Impossível de comparar

32. No código: `personas.sort(key=lambda x: x[1])`, o que essa lambda está fazendo?

- a) Ordenando pelo primeiro elemento de cada item
- b) Definindo que a ordenação deve usar o SEGUNDO elemento (índice 1) de cada item como critério
- c) Removendo o segundo elemento de cada item
- d) Somando os elementos de cada item

Módulo 6 — Tratamento de Exceções

33. Qual é a diferença fundamental entre um erro de sintaxe e uma exceção?

- a) São a mesma coisa
- b) Erro de sintaxe impede o código de rodar; exceção ocorre em tempo de execução mesmo com o código correto
- c) Exceção só ocorre em arquivos
- d) Erro de sintaxe só ocorre em loops

34. Qual exceção é gerada ao tentar dividir um número por zero?

- a) `ValueError`
- b) `ZeroDivisionError`

- c) TypeError
- d) KeyError

35. Qual bloco em um try/except SEMPRE é executado, tenha ocorrido erro ou não?

- a) else
- b) except
- c) finally
- d) raise

36. Para que serve a palavra reservada 'raise' em Python?

- a) Para capturar uma exceção
- b) Para disparar/lançar uma exceção manualmente, mesmo sem erro nativo do Python
- c) Para finalizar o programa imediatamente
- d) Para importar um módulo

37. Qual exceção ocorre ao tentar acessar uma chave que não existe em um dicionário?

- a) IndexError
- b) KeyError
- c) AttributeError
- d) NameError

38. Por que capturas genéricas como 'except Exception' são consideradas uma má prática?

- a) Porque são proibidas pelo Python
- b) Porque podem mascarar erros inesperados e dificultar a depuração do código
- c) Porque tornam o código mais rápido demais
- d) Porque exigem o uso de logging obrigatoriamente

Módulo 7 — Arquivos

39. Qual a principal vantagem de usar 'with open(...) as arquivo:' em vez de 'open()' com '.close()' manual?

- a) É mais rápido para processar arquivos grandes
- b) Garante o fechamento automático do arquivo, mesmo se ocorrer um erro no meio
- c) Só funciona com arquivos binários
- d) Permite abrir arquivos sem informar o modo

40. Qual modo de abertura CRIA o arquivo se não existir e SOBRESCREVE todo o conteúdo se ele já existir?

- a) 'r'
- b) 'a'
- c) 'w'
- d) 'x'

41. Qual modo de abertura adiciona conteúdo ao FINAL de um arquivo já existente, sem apagar o que já estava lá?

- a) 'w'
- b) 'a'
- c) 'r'
- d) 'x'

42. Qual método de leitura retorna o conteúdo de um arquivo já organizado como uma LISTA de linhas?

- a) .read()
- b) .readline()
- c) .readlines()
- d) .write()

43. Ao abrir um arquivo de imagem ou vídeo, qual modo deve ser usado?

- a) 'r' (texto)
- b) 'rb' (leitura binária)

- c) 'a' (append)
- d) 'x' (criação)

44. Qual codificação é considerada o padrão universal mais usado na web, em Python e em JSON?

- a) ASCII
- b) UTF-16
- c) UTF-8
- d) Windows-1252

Módulo 8 — Gerenciamento de Módulos

45. O que é o pip em Python?

- a) Um tipo de dado nativo
- b) O gerenciador de pacotes padrão do Python, usado para instalar/atualizar/remover bibliotecas
- c) Um framework para criação de sites
- d) O nome do interpretador Python

46. Qual comando gera um arquivo requirements.txt com as dependências do projeto?

- a) pip install requirements.txt
- b) pip freeze > requirements.txt
- c) pip list requirements.txt
- d) pip build requirements.txt

47. Para que serve um ambiente virtual (venv) em Python?

- a) Para deixar o código mais rápido
- b) Para isolar as dependências de cada projeto, evitando conflitos de versão entre pacotes
- c) Para substituir o pip
- d) Para criptografar o código-fonte

48. Qual comando cria um novo ambiente virtual chamado 'meu_ambiente'?

- a) pip venv meu_ambiente
- b) python -m venv meu_ambiente
- c) venv --new meu_ambiente
- d) python create venv meu_ambiente

49. Qual arquivo indica que uma pasta deve ser reconhecida como um pacote Python?

- a) README.md
- b) setup.py
- c) __init__.py
- d) LICENSE

50. Qual é o repositório oficial de pacotes de software da comunidade Python?

- a) GitHub
- b) PyPI (Python Package Index)
- c) npm
- d) Docker Hub

GABARITO

Confere aí como foi! Cada resposta certa é um degrau a mais rumo à prova. 💪

1-B	2-B	3-B	4-B	5-B
6-B	7-C	8-B	9-C	10-B
11-B	12-B	13-B	14-B	15-B
16-A	17-B	18-B	19-C	20-B
21-B	22-C	23-C	24-B	25-B
26-B	27-B	28-B	29-B	30-A
31-C	32-B	33-B	34-B	35-C
36-B	37-B	38-B	39-B	40-C
41-B	42-C	43-B	44-C	45-B
46-B	47-B	48-B	49-C	50-B

Boa prova! Você já leu, organizou e testou o conteúdo — agora é só confiar no preparo. 🎓✨